

# COMICCRAFT — AI COMIC STORY CREATOR

## 3. Project Design | Architecture, components, workflow, data & interface design

### SYSTEM ARCHITECTURE

**Frontend** → **FastAPI Backend** → **AI Services** → **Comic Preview / PDF Export**

**Frontend:** HTML, CSS, Jinja2 templates collect story prompt, character name, setting, tone and art style.

**Backend:** FastAPI routes receive/validate input, coordinate generation and render templates or JSON.

**AI layer:** Gemini Flash creates the structured 5-panel outline; Gemini Pro creates narration/dialogue; Stable Diffusion creates panel illustrations.

**Output:** generated panels are organized into a layout and exported as a downloadable PDF.

### INTERFACE DESIGN

- **Home page (index.html):** simple form for prompt, character, setting, tone and art style.
- **Comic Preview (comic\_preview.html):** panel title, AI image, scene description, caption/narration and image prompt reference.
- **Export Success (export\_success.html):** confirms PDF creation/download and offers a path to create another comic.
- **Responsive presentation:** clean centered layout, grouped inputs, rounded result panels and simple navigation.

### COMPONENT DESIGN

- **main application / routes.py:** FastAPI app and route orchestration.
- **gemini\_flash.py:** generate\_outline() for structured 5-panel outlines.
- **gemini\_pro.py:** generate\_story() for narration and character dialogue.
- **image\_generator.py:** generate\_image() for Stable Diffusion illustrations.
- **layout\_builder.py:** build\_comic\_layout() matches images with panel stories.
- **exporters.py:** save\_pdf() creates the multi-page comic PDF.
- **templates/:** index.html, comic\_preview.html, export\_success.html for the user interface.

### GENERATION WORKFLOW

1. **Input** → user enters story details
2. **Outline** → Gemini Flash generates a structured 5-panel outline
3. **Story** → Gemini Pro expands panels into narration/dialogue
4. **Images** → Stable Diffusion generates an illustration for each panel
5. **Layout** → layout\_builder.py combines images and story content
6. **Preview** → Jinja2 renders the comic panel-by-panel
7. **Export** → FPDF creates the downloadable multi-page PDF

### TECHNOLOGY STACK & DEPLOYMENT

FastAPI + Uvicorn • Python • HTML/CSS/Jinja2 • Google Gemini API • Hugging Face Diffusers / Stable Diffusion • PyTorch • FPDF • Pillow • Git / virtual environment.  
Local run documented with `uvicorn app.main:app --reload`; API documentation is available through FastAPI's `/docs` route.

### DATA DESIGN

**User input data:** story prompt, main character name, setting, story tone, art style.

**Panel data:** panel number, title, scene description, image-generation prompt, captions/narration and dialogue.

**Generated assets:** panel images, structured layout data and final PDF path/file.

The current project documentation describes generation and export data; it does not define a persistent user/account database.

**Project Design Summary:** ComicCraft uses a three-layer web architecture—frontend, FastAPI backend, and AI integration—to transform a user prompt into a complete illustrated comic and downloadable PDF. The modular design separates outline generation, story generation, image generation, layout building and export.